

Enterprise Doc AI

**A Local-First Retrieval-Augmented Generation System
for Private Document Question Answering**

Shahab Azmi

May 2026

Motivation

- Hosted LLMs cannot be trusted with confidential material – contracts, exam papers, internal HR documents.
- Generic LLMs hallucinate; hosted RAG services are opaque about *which passages* grounded an answer.
- Paid APIs are metered: every page indexed and every query has a unit cost.

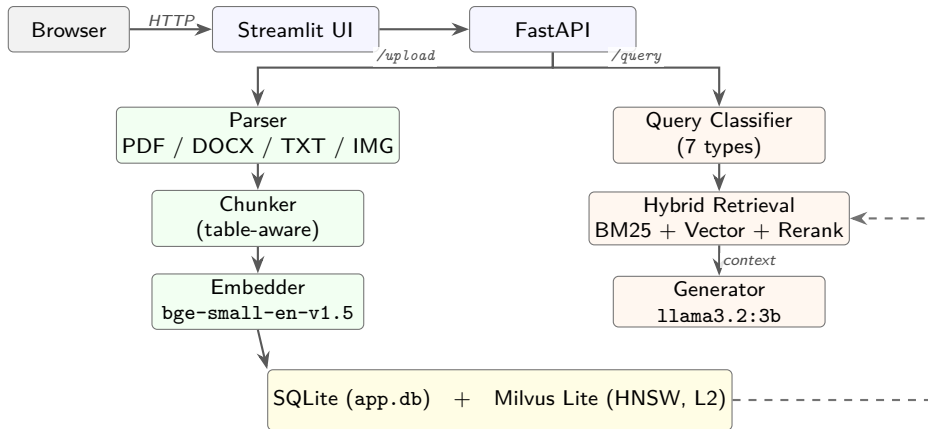
Thesis

Push RAG to its local-first limit. Parser, embedder, BM25 index, vector store, generator and conversation memory all run on a single Apple M2 (8 GB RAM, no GPU, no internet at query time).

Contributions

1. A complete, runnable **local-first RAG pipeline** – BGE bi-encoder, Okapi BM25, and a 3 B Q4-quantised generator served by Ollama.
2. A **seven-class query classifier** that routes each query to a different prompt template, retrieval depth, and token budget. *Two of seven classes bypass the LLM entirely.*
3. A **hybrid reranker** fusing normalised BM25 and dense ranks, plus small domain-aware bonuses tuned for personal-document corpora.
4. A **table-aware chunker** that detects high-density numeric line patterns and preserves their structure for retrieval over financial PDFs.
5. A **persistence design** where SQLite is the single source of truth and the embedded vector store is rebuilt from it on startup.

System Architecture



Solid arrows = write; dashed = read. **Green**: ingest pipeline. **Orange**: query pipeline. SQLite is the single source of truth.

Implementation: Table-aware Chunking

Problem. Naive whitespace collapsing destroys tables – rows and columns merge into a single line, hurting both BM25 and vector retrieval scores.

Heuristic. If ≥ 3 lines exist and $> 40\%$ are short (< 80 chars) *and* contain a digit, treat the chunk as tabular: keep line breaks, collapse only intra-line whitespace.

```
def _normalize_chunk_text(text):
    lines = [l for l in text.split("\n") if l.strip()]
    numeric_short = sum(1 for l in lines
                        if len(l.strip()) < 80 and any(c.isdigit() for c in l)
                        )
    if len(lines) >= 3 and numeric_short / len(lines) > 0.4:
        return "\n".join(" ".join(l.split()) for l in lines)    # table mode
    return " ".join(text.split())                                # prose mode
```

Added after observing that *Tesla, Inc. Form 10-K* pages produced unreadable single-line chunks before normalisation – which severely hurt retrieval over the 633-chunk financial PDF.

Hybrid Retrieval

Four-stage pipeline (backend/services/retrieval.py):

1. Stop-word removal + alias expansion (professor $\rightarrow$ {prof, instructor, lecturer, teacher}).
2. **Lexical**: BM25 Okapi over chunk tokens – top $\max(3k, 12)$ candidates.
3. **Dense**: Milvus Lite (HNSW, L2), top $\max(4k, 16)$ candidates; L2 distance > 1.2 discarded as too weak.
4. **Rerank**:

$$\text{score}(d) = w_b \cdot \hat{b}(d) + (1 - w_b) \cdot \hat{r}(d) + \beta(q, d)$$

Domain bonuses (β , +0.10 – +0.18)

Page-1 boost for instructor queries; @-token bonus for contact queries; parenthetical-acronym bonus (“NLP” matches “Natural Language Processing (NLP)”); $w_b=0.7$ for NUMERICAL queries (exact terms outweigh semantics).

Query Classification: 7 Types

Type	k	w_b	n_{predict}	LLM?	Triggers
EXTRACTIVE	7	0.5	250	no	“exact paragraph”, “verbatim”
PROVENANCE	3	0.5	60	no	“which page”, “where in”
NUMERICAL	5	0.7	60	yes	“how many”, “percentage”
LIST	5	0.6	150	yes	“list all”, “enumerate”
REASONING	5	0.4	120	yes	“why does”, “analyse”
SUMMARY	6	0.5	300	yes	“summarise”, “key points”
FACTUAL	3	0.5	120	yes	default

LLM-bypass

EXTRACTIVE returns the top chunk verbatim; PROVENANCE returns only metadata (file / page / section). Skipping the generator removes 5–10s of latency and eliminates the risk of a 3 B model paraphrasing a quote.

Generation & Prompt Engineering

Single Ollama call per query, temperature = 0. Prompt = five-rule core + type-specific addendum.

Five-rule core

- Use only the retrieved context; reply “Not found in document.” when context is silent.
- Stop after one answer; prefer exact wording; cap at 3–4 sentences unless asked otherwise.

Type addenda block specific failure modes – e.g. REASONING forbids hedge phrases (“could lead to”, “analysts believe”) the model would otherwise invent.

Empirical finding

The retrieved context is concatenated with a **3000-character cap**. Filling the model’s full ~4 k context window with chunks *degraded* answer quality on the 3 B model – a longer context was a worse context, not a better one.

Evaluation

Corpus (660 chunks total)

Document	Chunks
Tesla 10-K (FY2024) PDF	633
NLP Course Syllabus PDF	16
AI Homework 1 PDF	7
Resume DOCX	4

Hardware. Apple M2, 8 GB unified memory, no GPU.

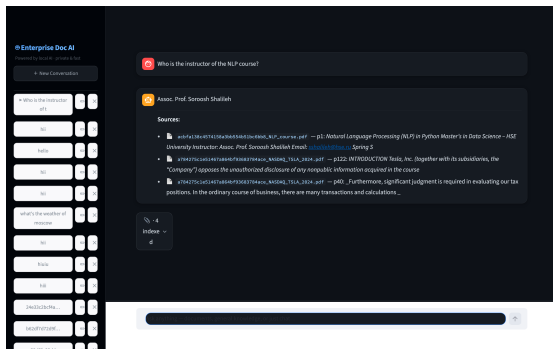
Latencies

Ingest 633-chunk PDF: ≈ 22 s.

Factual query: 5–8 s.

Summary: 12–18 s.

Cold-start LLM load: ≈ 3 s once.



Grounded answer to a **FACTUAL** query, with three source citations (file / page / snippet) rendered from chunk metadata.

Limitations & Future Work

Limitations

- **3 B generator.** llama3.2:3b occasionally drops a correct retrieved fact in favour of a near-paraphrase; a 7–13 B model would lift the answer-quality ceiling.
- **No cross-encoder rerank.** Hybrid lexical/dense fusion approximates it cheaply, but is weaker.
- **Rule-based router** can misclassify document queries that look superficially like “ask the world” questions.

Future work

1. **Cross-encoder rerank** (bge-reranker-base over top-20) – likely the single largest retrieval-quality lift.
2. **Streaming responses** – Ollama token-stream → FastAPI SSE → Streamlit.
3. **Learned router** replacing the current rule-based router.

Conclusion

- A fully **local-first** RAG system runs end-to-end on commodity laptop hardware, with no cloud or paid-API dependency.
- The engineering value lies in *integration*: type-aware routing, LLM-bypass for the cheapest question types, table-aware chunking, and SQLite-as-source-of-truth.
- Result: a system a single user can run, understand, and **extend on their own hardware** – the local-first counterpart to hosted RAG products.

Thank you. Questions?

github.com/shahabazmi/intelligent-doc-qa